

報告文件

一、 Periodic task set 產生方式說明：

本專案使用 Python 撰寫 task_generator.py 來產生 periodic task set。產生器的目標是自動建立符合題目限制的 task set，並輸出成指定的 JSON 格式：

F. Periodic task set 範例

```
{
  "periodic": {
    "p1": {"r": 1, "p": 12, "e": 3, "d": 10, "w": 15, "preempt": 1},
    "p2": {"r": 3, "p": 20, "e": 4, "d": 15, "w": 10, "preempt": 1},
    "p3": {"r": 5, "p": 24, "e": 4, "d": 18, "w": 12, "preempt": 0}
  }
}
```

欄位說明：

Key	Description
r	release time
p	Period
e	execution time
d	relative deadline
w	energy demand, unit: MWh
preempt	1 = preemptable, 0 = non-preemptable

每一個 periodic task 會包含 release time r、period p、execution time e、relative deadline d、energy demand w，以及是否可搶佔的 preempt 欄位，其中 preempt = 1 代表 preemptive，preempt = 0 代表 non-preemptive。

產生方式並不是完全隨機產生後再檢查，因為此方法容易產生大量不合法的 task set，導致執行時間過長。因此本產生器採用「限制導向」的建構方式。程式會先決定 task 數量，再產生 period、execution time、deadline、energy demand 與 preemptive 屬性，並在產生過程中控制重要條件，使 task set 較容易直接滿足限制式。

在 workload density 的部分，產生器使用以下公式計算：

$$DW = \sum(e_j / \text{period}_j)$$

並限制其範圍為：

$$0.7 \leq DW \leq 1.1$$

其中 0.7 是題目要求的下限，1.1 是本專案額外加入的上限限制，避免產生過度密集、可能難以排程的 task set，以至於後續無法插入 aperiodic 或 sporadic jobs。

為了確保 frame size 存在，產生器預設使用 $f=4$ 作為可行 frame size 的建構目標，並保證其滿

足：

$$f \geq \max(e_j)$$

$$72 \bmod f = 0$$

$$2f - \gcd(f, \text{period}_j) \leq \text{deadline}_j$$

因此在產生 deadline 時，程式會根據 period 與 frame size 計算 deadline 的下限，避免產生之後無法通過 frame size 限制的 task set。

此外，產生器也會保證以下條件：

- periodic task 數量介於 6 到 10 個
- 72 小時展開後的 job 數量大於 30
- period 介於 6 到 24，且至少包含 3 種不同 period
- execution time 介於 1 到 4
- 至少 2 個 task 的 $e = 2$
- 至少 1 個 task 的 $e \geq 3$
- deadline 滿足 $e \leq d \leq p$
- energy demand 介於 6 到 18
- 至少 2 個 task 的 $w \geq 14$
- 至少 20% 的 task 滿足 $d = e$
- 至少 2 個 $e \neq 1$ 的 task 為 non-preemptive

task set 產生完成後，程式會再透過 validator 進行完整檢查。只有所有限制式皆通過時，才會輸出 JSON 檔案。因此最後產生的 periodic_task_set.json 可以保證符合題目對 periodic task set 的所有限制。

註：本產生器選擇 frame size $f = 4$ ，原因是 execution time 的限制為 $1 \leq e_j \leq 4$ ，因此 f 必須至少大於等於所有 task 的最大 execution time，也就是 $f \geq \max(e_j)$ 。由於本產生器會產生 $e_j = 4$ 的 task，因此 frame size 至少需要為 4。

另外，frame size 也必須滿足 $72 \bmod f = 0$ ，因此 f 必須是 72 的因數。在所有符合 $f \geq 4$ 且可整除 72 的候選值中， $f = 4$ 是最小的 frame size。選擇較小的 frame size 可以讓 $2f - \gcd(f, \text{period}_j) \leq \text{deadline}_j$ 這個限制較容易被滿足，因為 f 越大，左式通常也會越大，deadline 條件會變得更嚴格，在實際操作中，其他的 framesize 容易造成執行時間過長甚至無解，不符效益。

因此，本產生器固定使用 $f = 4$ 作為目標 frame size，並在產生 deadline 時確保所有 task 都滿足 frame size 限制。

二、 放寬 Assumption 的限制式建模

本作業 Level 2 在 Level 1 模型基礎上放寬三項 assumption，分別為再生能源不確定性、儲能設備老化成本，以及彈性市場機制。Level 2 未新增決策變數，仍沿用 Level 1 的 $P_{i,t}$ 、 $k_{j,i,t}$ 、 $Sell_t$ 與 $SOC_{i,t}$ ；新增內容主要為參數、限制式與目標函數成本項。

1. 再生能源不確定性

Level 1 假設再生能源實際出力等於日前預測值。Level 2 放寬此 assumption，考慮實際再生能源出力可能低於或不同於 forecast。

新增 notation：

$\rho_{i,t}$ ：再生能源 i 在時間 t 的實際出力修正倍率

$renewactual_{i,t}$ ：再生能源 i 在時間 t 的實際可用出力比例

限制式：

$$renewactual_{i,t} = renewforecast_{i,t} * \rho_{i,t}$$

$$0 \leq renewactual_{i,t} \leq 1$$

$$0 \leq P_{i,t} \leq renewmax_i * \Delta t * renewactual_{i,t},$$

$$\forall i \in I_r, \forall t \in T$$

其中 $\rho_{i,t}$ 用來模擬天氣、雲量或其他不確定因素造成的再生能源實際出力變化。此限制式確保排程中使用的再生能源不會超過該時段實際可用量。

2. 儲能設備老化成本

Level 1 僅考慮儲能設備的 SOC 上下限、最大充放電功率，以及不可同時充放電。Level 2 進一步考慮電池充放電造成的老化成本。

新增 notation：

$Throughput_i$ ：儲能設備 i 在排程期間的充放電總量

$cdeg_i$ ：儲能設備 i 每 MWh throughput 的老化成本

$Cost_{deg}$ ：總儲能老化成本

儲能 throughput 定義：

$$Throughput_i =$$

$$\sum_t P_{i,t}$$

+

$$\sum_t \sum_{\{j \in J_{chg} : b(j)=i\}} \sum_{\{s \in I_g \cup I_r\}} k_{j,s,t}$$

其中第一項 $\sum_t P_{i,t}$ 表示儲能設備放電總量，第二項表示充入該儲能設備的總電量。

老化成本：

$$Cost_{deg} = \sum_{\{i \in I_b\}} cdeg_i * Throughput_i$$

此成本項不改變原本 SOC 限制式，而是將儲能設備使用量轉換為額外成本，使排程不會將電池視為完全免費且無損耗的資源。

3. 彈性市場機制

Level 1 假設售電價格固定且使用日前市場價格。Level 2 放寬此 assumption，考慮即時市場價格變動，以及日前售電承諾未達成時的 penalty。

新增 notation：

λ_{RT_t} ：時間 t 的即時市場價格

$SellDA_t$ ：Level 1 日前排程中時間 t 的售電承諾量

$Shortfall_t$ ：時間 t 的售電短缺量

γ ：每 MWh 售電短缺 penalty

$Penalty_sell$ ：總售電短缺 penalty

即時市場收益：

$$Revenue_RT = \sum_t \lambda_{RT_t} * Sell_t$$

售電短缺量：

$$Shortfall_t = \max(0, SellDA_t - Sell_t)$$

售電短缺 penalty：

$$Penalty_sell = \sum_t \gamma * Shortfall_t$$

其中 $SellDA_t$ 由 Level 1 靜態日前排程產生，代表系統原先承諾售出的電量。若 Level 2 因實際再生能源出力不足、突發 job 需求或儲能狀態限制導致實際售電量下降，則會產生 shortfall penalty。

4. Level 2 目標函數

Level 1 目標函數為：

$$\min F = \alpha f_1 + f_2 + f_3$$

Level 2 加入儲能老化成本與售電短缺 penalty，並使用即時市場收益計算售電收入：

$$\min F_L2 = \alpha f_1 + f_2 + f_3_RT + Cost_deg + Penalty_sell$$

其中：

$$f_1 = \sum_{j \in J_a} Miss_j$$

f_2 = 傳統機組發電成本

$$f_3_RT = - Revenue_RT$$

因此可寫為：

$$F_L2 =$$

$$\alpha * AperiodicMissCount$$

$$+ GeneratorCost$$

$$+ StorageAgingCost$$

$$+ SellShortfallPenalty$$

$$- RealTimeMarketRevenue$$

此目標函數反映 Level 2 中 deadline performance、傳統機組成本、儲能使用成本與市場收益之間的權衡關係。

三、 排程演算法設計說明

(1) 日前排程演算法設計說明

本系統採用 72 小時日前靜態排程，排程時間單位為 1 小時，frame size 設定為 4。日前排程主要由 scheduler.py 負責，流程包含 periodic jobs 展開、合法 frame 檢查、job time-slot 排程、能源調度，以及結果驗證。

1. Periodic jobs 展開

系統先讀取 output/task_set.json 中的 periodic task set。每個 periodic task 包含 release time、period、execution time、relative deadline、energy demand，以及 preemptive / non-preemptive 屬性。

接著根據 task 的 period，在 72 小時 horizon 內展開成多個 job instances，例如 p1_1、p1_2 等。每個 job 會計算其 absolute deadline：

$$\text{absolute_deadline} = \text{release_time} + \text{relative_deadline} - 1$$

若展開後的 job deadline 超出 72 小時範圍，則不納入排程並記錄為 skipped job。

2. Frame size 合法性檢查

日前排程使用固定 frame size：

$$f = 4$$

程式會檢查：

$$H \bmod f = 0$$

$$f \geq \max(e_j)$$

$$2f - \gcd(f, p_j) \leq d_j$$

若任一條件不符合，代表該 task set 不適合此 frame size，排程會被視為不合法。

3. Job time-slot 排程

對每個 job，系統會根據 release time 到 absolute deadline 之間的可執行區間產生候選時段。

若 job 是 non-preemptive，候選時段必須連續：

$$[t, t+1, \dots, t+e_j-1]$$

若 job 是 preemptive，則允許在 release-deadline window 內分散執行，只要累積執行時間等於 execution time。

系統接著使用 backtracking search 進行排程。每次選擇目前可行候選數最少的 job 優先安排，以降低搜尋空間。排程過程中會維護每小時已安排的用電負載，確保：

$$\text{scheduled_load}_t + w_j \leq \text{load_capacity}_t$$

因此每個 periodic job 都必須在 release time 之後、absolute deadline 之前完成，並且不能超過該時段的供電容量。

4. 時段選擇評分

候選時段會根據 hour score 排序。hour score 綜合考慮市場價格與再生能源可用量，使排程傾向選擇較適合執行 job 的時間。

概念上，價格高的時段較適合保留能源售電，而再生能源較多的時段較適合安排負載，以降低傳統

機組成本。

5. 能源調度

當 job 的執行時段決定後，系統進一步建立每小時能源分配。能源調度順序如下：

1. 優先使用再生能源供應用電需求。
2. 若再生能源不足，使用儲能設備放電。
3. 若仍有不足，使用傳統機組補足 residual load。
4. 若再生能源有剩餘，且儲能設備尚有容量，則可安排充電。
5. 若總供電量仍有剩餘，則以 sell 形式售電到市場。

傳統機組調度會考慮：

output_min / output_max

ramp_up_rate / ramp_down_rate

min_up_time / min_down_time

fixed cost

variable cost

儲能設備調度會考慮：

soc_min / soc_max

charge_max / discharge_max

不可同時充電與放電

SOC transition

6. 輸出結果

日前排程完成後，系統輸出 schedule_result.json。每個時間點包含：

t

frame_id

running_jobs

P

k

sell

soc

missed_aperiodic

rejected_sporadic

其中：

- P 表示每個 processor 在該時段的供電量。
- k 表示每個 job 從各 processor 獲得的電量。
- sell 表示售電量。
- soc 表示儲能設備剩餘電量。

7. 可行性驗證

最後系統會檢查排程是否滿足主要限制式，包括：

- 所有 periodic jobs 是否完整執行。
- hard deadline jobs 是否皆準時完成。
- non-preemptive jobs 是否連續執行。
- 每小時能源是否平衡。
- 傳統機組是否滿足出力、ramp、min up/down 限制。
- 再生能源出力是否不超過可用量。
- 儲能設備是否滿足 SOC、充放電上限、不可同時充放電限制。

本日前排程演算法的目標是在滿足所有 periodic hard-deadline jobs 的前提下，保留一定資源餘裕供後續 sporadic jobs acceptance test 使用，並兼顧傳統機組成本與售電收益。

(2)進階動態排程方法設計說明

Level 2 的進階動態排程由 advanced_scheduler.py 負責。此方法建立在 Level 1 日前排程模型上，進一步考慮實際再生能源出力、即時市場價格、儲能老化成本與日前售電承諾 penalty。整體方法採用 **event-triggered rolling refresh**，也就是當系統狀態出現明顯變化時，更新排程依據並重新產生 Level 2 排程結果。

1. 動態資訊建模

Level 2 先根據原本的 processor_settings.json 與 price_72hr.json 建立動態狀態：

$\text{actual renewable availability} = \text{forecast renewable availability} * \rho_{i,t}$

其中 $\rho_{i,t}$ 表示再生能源實際出力倍率，用來模擬實際再生能源和日前預測不同。

同時，Level 2 會根據 real-time price multiplier 產生即時市場價格：

$\text{lambdaRT}_t = \text{lambda}_t * \text{price_multiplier}_t$

因此排程不再只依賴 Level 1 的固定 forecast 與 day-ahead price。

2. 更新觸發條件

進階動態排程採用 event-triggered rolling refresh。更新觸發條件包含：

$t \in \{1, 24, 48\}$

代表固定 rolling checkpoint。

$\text{forecast renewable availability} - \text{actual renewable availability} > 5 \text{ MWh}$

代表實際再生能源明顯低於日前預測。

$|\text{real-time price} - \text{day-ahead price}| > 10$

代表市場價格變動明顯，可能影響售電收益與能源使用決策。

當任一條件成立時，系統會記錄一筆 dynamic update event，內容包含 forecast renewable、actual renewable、renewable gap、day-ahead price、real-time price 與 trigger reason。

3. 動態排程流程

當 Level 2 狀態建立後，系統會執行以下流程：

1. 重建 Level 1 baseline，作為 day-ahead sell commitment 與比較基準。
2. 使用 actual renewable availability 取代 forecast renewable availability。

3. 使用 real-time price 取代 day-ahead price。
4. 重新計算 hour score 與 load capacity。
5. 展開 periodic jobs，並使用 scheduler 核心重新安排 hard-deadline periodic jobs。
6. 對 sporadic jobs 執行 acceptance test。
7. 對 aperiodic jobs 執行 soft-deadline scheduling。
8. 建立每小時 P、k、sell、soc 排程結果。
9. 檢查 hard deadline、energy balance、generator constraints、renewable constraints 與 storage constraints。
10. 輸出正式 schedule_result.json、acceptance_test_log.json 與 evaluation_results.json。

4. Sporadic 與 Aperiodic 處理

Sporadic jobs 屬於 hard deadline jobs。當 sporadic job 到達時，系統會在不破壞既有 periodic jobs 與已接受 hard-deadline jobs 的前提下進行 acceptance test。若 release time 到 deadline 之間有足夠空檔與能源容量，則接受並安排；否則拒絕並記錄原因。

Aperiodic jobs 屬於 soft deadline jobs，不進行 hard-deadline acceptance test。系統會優先安排在 soft deadline 前；若 deadline 前無法完成，仍可安排於 deadline 後，但會記錄 soft deadline miss 與 tardiness。

5. Level 2 額外成本計算

進階動態排程完成後，Level 2 會額外計算兩項成本。

儲能老化成本：

$\text{StorageAgingCost} = \text{storage throughput} * \text{aging cost per MWh}$

售電承諾短缺 penalty：

$\text{Shortfall}_t = \max(0, \text{SellDA}_t - \text{Sell}_t)$

$\text{Penalty}_{\text{sell}} = \sum_t \gamma * \text{Shortfall}_t$

其中 SellDA_t 來自 Level 1 baseline，表示日前售電承諾量。

Level 2 目標函數為：

$F_{L2} =$

$\alpha * \text{AperiodicMissCount}$

$+ \text{GeneratorCost}$

$+ \text{StorageAgingCost}$

$+ \text{SellShortfallPenalty}$

$- \text{RealTimeMarketRevenue}$

6. 方法選擇理由與 Tradeoff

採用 event-triggered rolling refresh 的原因是，不需要每小時都進行完整重排，可降低計算成本；但當再生能源實際出力或市場價格與日前資料差異明顯時，系統仍能更新排程依據，避免完全沿用過時資訊。

此方法的 tradeoff 包含：

- 若為了提高 sporadic job 接納率或維持 hard deadline feasibility，可能需要增加傳統機組發電或使用更多儲能，導致 generator cost 與 storage aging cost 上升。
- 若追求 market revenue，可能會增加售電量，但也可能降低可用 reserve，影響突發 jobs 的接納能力。
- 若減少重排頻率，可降低計算成本，但對動態資訊的反應會比較慢。
- 若頻繁重排，可更即時反應狀態變化，但計算成本較高，也可能使排程穩定性下降。

7. 結果正確性

本次 Level 2 排程結果中：

hard deadline miss rate = 0.0

soft deadline miss rate = 0.0

sporadic value rate = 1.0

constraint violations = 0

代表 hard-deadline jobs 皆於期限前完成，aperiodic jobs 未發生 soft miss，sporadic jobs 皆成功完成，且能源平衡、傳統機組限制、再生能源限制與儲能 SOC 限制皆通過 evaluator 檢查。

若後續 demo input 較嚴格而發生 sporadic rejection、deadline miss 或重排失敗，系統會在 acceptance_test_log.json 與 evaluation_results.json 中記錄原因與影響。

四、效能分析

(1) 日前排程效能分析

本系統的日前排程以 72 小時為排程範圍，先完成所有 periodic hard-deadline jobs 的靜態排程，再保留部分資源餘裕供後續 sporadic jobs acceptance test 使用。效能分析主要從 deadline 表現、response time、sporadic acceptance、aperiodic miss、成本與售電收益，以及限制式可行性進行評估。

1. Deadline 表現

日前排程首先確保所有 periodic jobs 在各自的 absolute deadline 前完成。排程完成後，系統會計算 hard deadline miss rate。Hard deadline jobs 包含 periodic jobs 與 accepted sporadic jobs。

本次結果中：

hard deadline miss rate = 0.0

代表所有 hard-deadline jobs 均在 deadline 前完成，沒有發生 hard deadline miss。這表示日前排程在滿足 periodic jobs 時限要求方面是可行的。

2. Response Time

Response time 定義為：

$$R_j = C_j - r_j$$

其中 C_j 為 job completion time， r_j 為 job release time。

本系統在排程時會根據 hour score 對候選時段排序，並盡量在可行條件下縮短 response time。

Response time 越短，表示 job 從釋放到完成的等待時間越短，系統反應越快。

3. Sporadic Acceptance Test 效能

日前排程會保留部分 capacity 作為 reserve，供 sporadic jobs 到達後進行 acceptance test。當 sporadic job 到達時，系統會檢查其 release time 到 hard deadline 之間是否仍有足夠空檔與能源容量。若可行則接受，否則拒絕。

Sporadic 效能以 sporadic value rate 評估：

sporadic value rate =

deadline 前完成的 sporadic jobs execution time 總和

/

demo 提供的 sporadic jobs execution time 總和

本次結果中：

sporadic value rate = 1.0

表示所有 sporadic jobs 的 execution time 皆在 hard deadline 前完成，sporadic 排程效能良好。

4. Aperiodic Soft Deadline 表現

Aperiodic jobs 屬於 soft deadline jobs，不進行 hard-deadline acceptance test。系統會優先安排在 soft deadline 前；若無法準時完成，仍可安排於 deadline 後，但會記錄 soft miss 與 tardiness。

本次結果中：

soft deadline miss rate = 0.0

表示所有 aperiodic jobs 皆在 soft deadline 前完成，沒有產生 tardiness。

5. 成本與售電收益

日前排程除了滿足 job deadline，也會考量傳統機組成本與售電收益。傳統機組成本包含：

$\text{generator cost} = \text{fixed cost} + \text{variable generation cost}$

售電收益為：

$\text{market revenue} = \sum_t \text{market_price}_t * \text{Sell}_t$

排程演算法會根據價格與再生能源 availability 調整 job 安排與能源調度。當市場價格較高時，系統傾向保留更多能源售電；當再生能源較多時，則較適合安排負載或充電，以降低傳統機組發電成本。

6. 限制式可行性

最後，系統透過 evaluator 檢查所有主要限制式，包括：

- periodic / sporadic hard deadline 是否滿足
- non-preemptive jobs 是否連續執行
- 每小時能源平衡
- 傳統機組 output min/max
- ramp up / ramp down
- min up time / min down time
- renewable output 不超過可用量
- storage SOC 上下限
- storage charge/discharge max
- storage 不可同時充電與放電

本次結果中：

$\text{constraint violations} = 0$

表示日前排程結果滿足系統主要限制式，排程具有可行性。

7. 小結

整體而言，日前排程能在 72 小時內完成所有 periodic hard-deadline jobs，並保留足夠資源處理 sporadic 與 aperiodic jobs。本次結果 hard deadline miss rate、soft deadline miss rate 與 constraint violations 皆為 0，sporadic value rate 為 1.0，表示排程在 deadline 表現與系統可行性上皆達到良好效果。成本與收益方面，排程透過 renewable、storage 與 thermal generator 的調度，在滿足任務需求的同時兼顧傳統機組成本與市場售電收益。

(2)進階動態排程方法效能分析

Level 2 的進階動態排程以 Level 1 日前排程為基礎，加入 actual renewable availability、real-time price、storage aging cost 與 sell shortfall penalty。效能分析主要比較 Level 2 在動態情境下是否仍能維持排程可行性，並觀察 deadline 表現、sporadic 接納率、aperiodic miss、成本、收益與 objective value 的變化。

1. Deadline 與可行性表現

Level 2 使用 actual renewable availability 取代原本的 forecast renewable availability，因此再生能源可用量可能低於 Level 1 預測。即使如此，進階動態排程仍需確保 hard-deadline jobs 能準時完成。

本次 Level 2 結果中：

hard deadline miss rate = 0.0

soft deadline miss rate = 0.0

constraint violations = 0

表示 periodic jobs 與 accepted sporadic jobs 都在 hard deadline 前完成，aperiodic jobs 也沒有 soft miss。同時，能源平衡、傳統機組限制、再生能源實際可用量限制與儲能 SOC 限制皆通過 evaluator 檢查。

2. Sporadic Job 接納表現

Sporadic jobs 屬於 hard deadline jobs，Level 2 會在更新 renewable availability 和 real-time price 後重新執行 acceptance test。

本次結果中：

sporadic value rate = 1.0

表示 demo 中所有 sporadic jobs 的 execution time 都在 hard deadline 前完成。這代表即使考慮實際再生能源變動，系統仍保留足夠資源處理突發 hard-deadline jobs。

3. Aperiodic Job 表現

Aperiodic jobs 屬於 soft deadline jobs。進階動態排程會根據更新後的能源狀態與價格分數安排 aperiodic jobs，並優先安排於 soft deadline 前。

本次結果中：

soft deadline miss rate = 0.0

表示所有 aperiodic jobs 皆於 soft deadline 前完成，沒有產生 tardiness。

4. 動態更新觸發效能

Level 2 採用 event-triggered rolling refresh。觸發條件包含：

$t = 1, 24, 48$

$\text{forecast renewable} - \text{actual renewable} > 5 \text{ MWh}$

$|\text{real-time price} - \text{day-ahead price}| > 10$

此設計避免每個小時都重排，降低不必要的計算成本；但當 renewable shortfall 或價格變動明顯時，系統仍會更新排程依據。

因此，該方法在「動態反應」與「計算成本」之間取得平衡。

5. 成本與收益分析

Level 2 額外考慮真實運作成本與市場變動。主要指標包含：

generator cost

market revenue

storage aging cost

sell commitment penalty

objective value

其中：

- generator cost 表示傳統機組發電成本。當 actual renewable output 低於 forecast 時，需要更多 thermal generator 補足缺口，因此成本可能上升。
- market revenue 表示即時市場售電收益。由於 Level 2 使用 real-time price，收益會隨即時價格變動。
- storage aging cost 表示電池充放電 throughput 造成的老化成本。
- sell commitment penalty 表示實際售電量低於 Level 1 day-ahead commitment 時的短缺 penalty。
- objective value 綜合反映 aperiodic miss penalty、generator cost、storage aging cost、sell shortfall penalty 與 market revenue。

Level 2 目標函數為：

$F_{L2} =$

$\alpha * \text{AperiodicMissCount}$

$+ \text{GeneratorCost}$

$+ \text{StorageAgingCost}$

$+ \text{SellShortfallPenalty}$

$- \text{RealTimeMarketRevenue}$

6. 與 Level 1 的比較

Level 1 使用 forecast renewable availability 與 day-ahead price，因此排程較穩定且成本較容易預估。

Level 2 則進一步考慮 actual renewable availability 和 real-time price，排程結果更貼近現實，但成本與收益也會產生變化。

Level 2 相較 Level 1 的主要影響如下：

- 若 actual renewable output 低於 forecast，thermal generator 需補足缺口，generator cost 可能上升。
- 若 real-time price 高於 day-ahead price，market revenue 可能上升。
- 因加入 storage aging cost，電池使用量越高，objective value 會增加。
- 若 Level 2 實際售電量低於 Level 1 commitment，會產生 sell shortfall penalty。
- 即使成本變化，Level 2 仍需維持 hard deadline feasibility。

本次結果中，Level 2 仍維持：

hard deadline miss rate = 0.0

soft deadline miss rate = 0.0

sporadic value rate = 1.0

constraint violations = 0

表示 Level 2 在更嚴格的實際情境下仍保持排程可行性。

7. Tradeoff 討論

進階動態排程存在多個 tradeoff：

- 為了維持 hard deadline 可行性或提高 sporadic job 接納率，可能需要增加 thermal generator 發電，導致 generator cost 上升。
- 若使用更多 storage 來補足 renewable shortfall，會降低 thermal cost，但會增加 storage aging cost。
- 若追求較高 market revenue，可能會增加售電量，但可用 reserve 下降，可能降低未來 sporadic jobs 的接納能力。
- 若頻繁重排，可更即時反映 renewable 和 price 變化，但計算成本較高；若降低更新頻率，計算成本較低，但對動態資訊的反應較慢。

8. 小結

整體而言，Level 2 進階動態排程能在 actual renewable output 與 real-time price 變動下維持 deadline 與系統限制式可行性。雖然因 renewable shortfall、storage aging cost 和 sell shortfall penalty 使成本結構更複雜，但該方法能反映更真實的能源系統運作情境，並透過 event-triggered rolling refresh 在計算成本與即時反應能力之間取得平衡。

五、 討論與心得：

討論：

(1) task_generator.py

工具名稱： Gemini (透過與 AI 協作者深度對話進程式碼迭代)

在開發 Task Generator 的過程中，我們採取了「多階段漸進式迭代」的協作模式。由於即時系統 (Real-Time Systems) 的排程約束非常嚴苛且互相衝突，若直接使用傳統的隨機碰撞法 (Brute-force)，合格率近乎於零。我們的協作分為以下三個主要階段：

• 階段一：盲點突破與數學可行性分析

一開始嘗試盲目將 Workload Density (總利用率 DW) 設得過高 (大於 2)，經由與 AI 討論後，確認在單處理器系統中 $DW > 1.0$ 屬於不合法的排程。隨後進一步發現，作業中「20% 任務 deadline = e」以及「大魔王 Frame size 公式 ($2f - \gcd(f, p_j) \leq d_j$)」存在極強的數學約束衝突。

• 階段二：設計思路翻轉 (逆推思維)

為了防止 Generator 陷入無窮迴圈，與 AI 共同決定「直接鎖定最完美的 Frame Size ($f=4$)」，以此答案為核心，倒推 Period 的安全候選池 (皆為 4 的倍數)，從根本上閹割了大魔王公式的殺傷力，將程式的生成成功率提升至 100%。

• 階段三：全面覆核評分指標與格式微調

在最後階段，透過提供助教的評分標準與範例截圖，精準要求 AI 將所有「軟硬性指標」全數納入過濾器中 (如：任務總數 6~10 個、工作總數 >30、電力需求範圍、非搶占任務數量、特定任務長度等)，並自訂了 custom_json_format 輸出函數，完美還原助教講義中「一個 Task 佔據單獨一行」

的 JSON 規格。

3. Prompt 策略與實作技巧

本專案能夠成功在幾毫秒內產出完美測資，關鍵在於使用了以下幾種 **Prompt 策略**：

- **策略一：提供核心公式與脈絡限制 (Context-Setting)**

實作方法： 直接將作業說明中的關鍵限制（如 $DW \geq 0.7$ 、超週期 72、Frame Size 公式）餵給 AI。這有助於 AI 在第一時間指出「利用率大於 1 根本排不出來」的系統本質問題，避免走彎路。

- **策略二：多條件約束的優先級排序 (Constraint Prioritization)**

實作方法： 當發現條件太多（Period、Execution time、Energy 各有規矩）導致程式難以生成時，採取了「先抓大魔王，再微調普通任務」的 Prompt 指導原則。先強迫前四個核心任務滿足緊湊、非搶占、高電力限制，剩下的 5 號之後任務再做隨機分配，藉此在真隨機與硬指標之間取得平衡。

- **策略三：拒絕「硬改 (Hardcoding)」，堅持「重抽過濾 (Re-sampling)」**

實作方法： 在處理電力需求 $6 \leq w_j \leq 18$ 且至少兩個大於 14 的限制時，原本的 Prompt 寫法容易引導 AI 去手動硬改固定任務的電力數值，這會破壞資料的統計隨機性。隨後調整 Prompt 策略，要求 AI 實作「不合規就 continue 整組重抽」的過濾器機制，在 0.115 秒的高效率下，同時保有完美的隨機分佈與資料正確性。

- **策略四：視覺化與格式對齊 (Few-Shot Formatting)**

實作方法： 透過提供助教的 JSON 範例圖片，明確要求 AI 克服 Python 內建 `json.dump()` 會強制把每個 Key 拆成一行的缺陷，撰寫專屬的字串格式化器，確保輸出與作業規範分毫不差。

(2) 本專案中，`scheduler.py` 與 `evaluator.py` 是整體系統能否正確運作的核心。`scheduler.py` 負責產生實際排程結果，而 `evaluator.py` 則負責檢查排程結果是否符合限制式並計算效能指標。兩者的分工讓系統不只可以產生 schedule，也能進一步驗證該 schedule 是否合理，兩者皆使用 `chatgpt codex` 實作

Scheduler 討論

`scheduler.py` 的主要功能是將 `task_set.json` 中的 periodic tasks 展開為 72 小時內的 periodic jobs，並在固定 `frame size = 4` 的設定下安排所有 jobs 的執行時段。排程時，系統會考慮 `release time`、`absolute deadline`、`execution time`、`preemptive / non-preemptive` 屬性，以及每小時可用的能源容量。對 `non-preemptive jobs`，`scheduler` 會確保其執行時段連續；對 `preemptive jobs`，則允許分散到不同時段執行，只要累積執行時間滿足 `execution time` 並在 `deadline` 前完成即可。

在完成 periodic jobs 的 static schedule 後，`scheduler` 也加入了 sporadic 與 aperiodic jobs 的處理。Sporadic jobs 屬於 hard deadline jobs，因此 `scheduler` 會執行 acceptance test。Acceptance test 的原則是不移動已經排好的 periodic jobs，也不破壞已接受的 hard deadline jobs。若 sporadic job 可以在 `release time` 與 `deadline` 之間找到可行時段，且加入後不違反能源容量限制，則接受該 job；否則拒絕並記錄原因。為了避免 sporadic jobs 佔用過多未來資源，`scheduler` 也加入 reserve-aware 的概念，優先尋找保留部分容量後仍可行的排程位置。

Aperiodic jobs 屬於 soft deadline jobs，因此 `scheduler` 不會像 sporadic jobs 一樣進行 hard-deadline

acceptance test，而是將其視為可等待的 jobs。排程時，系統會優先嘗試將 aperiodic jobs 安排在 soft deadline 前；若無法在 deadline 前完成，則仍可安排在 deadline 後，但會記錄 soft deadline miss 與 tardiness。這樣的設計符合 aperiodic jobs 可逾期但需計算效能損失的特性。

除了 job timing 之外，scheduler 也負責 energy dispatch。每小時的排程結果包含各 processor 的供電量 P 、各 job 從不同 processor 獲得的電能 k 、售電量 $sell$ 與儲能設備狀態 soc 。在 dispatch 過程中，系統會考慮傳統機組的 output min/max、ramp-up/ramp-down、minimum up/down time，再生能源的 forecast cap，以及儲能設備的 SOC 上下限與充放電限制。這使得 schedule result 不只是 jobs 的時間配置，也包含可行的能源供需分配。

整體而言，scheduler 採用的是 feasibility-first heuristic，而不是全域最佳化方法。也就是說，系統優先確保 hard deadline jobs 能準時完成並滿足所有限制式，再進一步考量 aperiodic miss、傳統機組成本與售電收益。這樣的設計雖然不保證得到數學上的最佳解，但具有計算速度快、邏輯清楚、容易驗證與適合 demo 的優點。

Evaluator 討論

evaluator.py 的主要功能是讀取 scheduler 產生的 schedule_result.json 與 acceptance_test_log.json，重新計算各項效能指標，並輸出 evaluation_results.json。Evaluator 的存在可以避免只依賴 scheduler 自己輸出的 summary，而是從實際排程結果中再次檢查 deadline、response time、tardiness、成本、收益與限制式違反情形。

在 deadline 指標方面，evaluator 會區分 hard deadline jobs 與 soft deadline jobs。Periodic jobs 與 accepted sporadic jobs 會被視為 hard deadline jobs，用來計算 hard deadline miss rate。Aperiodic jobs 則根據 acceptance log 中的 miss 欄位計算 soft deadline miss rate，並進一步計算 average tardiness 與 max tardiness。這樣可以避免把 aperiodic jobs 的逾期完成誤判為 hard deadline violation，也符合題目對 aperiodic jobs 的定義。

在 response time 與 jitter 方面，evaluator 會根據每個 job 的 release time 與 completion time 計算 response time，並計算 average response time 與 max response time。對 periodic tasks，evaluator 也會計算 completion-time jitter，用來觀察同一個 periodic task 不同 instances 的完成時間是否穩定。雖然本系統主要優先考慮 deadline 與可行性，但 jitter 指標仍能反映 static schedule 的穩定程度。在能源與經濟指標方面，evaluator 會根據每小時傳統機組輸出量計算 generator cost，並根據市場價格與 sell quantity 計算 market revenue。Objective value 則使用 aperiodic miss penalty、generator cost 與 market revenue 組合而成。這使得 evaluator 可以同時呈現三個目標的結果：aperiodic miss 數量、傳統機組成本與售電收益。

此外，evaluator 也會檢查基本 constraint violations，例如每小時供需平衡、sell 是否非負、傳統機組輸出是否符合上下限與 ramp 限制、再生能源是否超過 forecast cap、儲能設備 SOC 是否在上下限內，以及是否出現同時充電與放電。若 constraint violation count 為 0，代表 schedule result 在 evaluator 的檢查下是可行的。

整體而言，scheduler 與 evaluator 的分工讓本系統形成一個完整流程。Scheduler 負責產生排程與能源配置，evaluator 則負責驗證與量化結果。這樣的設計有助於在開發過程中快速發現排程邏輯或限

制式處理上的錯誤，也能在報告中提供具體數據說明系統效能。

(3)本次作業在 Level 2 進階動態排程方法的設計與實作過程中使用 AI 工具輔助。使用的 AI 工具為 ChatGPT / Codex，主要協助內容包含理解作業規格、整理 Level 2 評分標準、設計放寬 assumptions 的建模方向、協助撰寫與修改 advanced_scheduler.py，以及協助檢查輸出 JSON 格式與 demo 說明內容。

在協作方式上，提供 Level 1 已完成的程式碼，包括 task_generator.py、scheduler.py 與 evaluator.py，並提供作業說明 PDF 與繳交資料夾結構。AI 先協助閱讀作業說明，整理 Level 2 所需完成的項目，接著根據既有 Level 1 程式架構，設計一個不破壞原本輸出格式的 Level 2 延伸方式。實作上，AI 協助新增 advanced_scheduler.py，使其能在 Level 1 scheduler 的基礎上加入 actual renewable availability、real-time price、storage aging cost 與 sell shortfall penalty，並輸出正式的 schedule_result.json、acceptance_test_log.json 與 evaluation_results.json。

Prompt 策略方面，主要採用逐步拆解與反覆修正的方式。首先詢問 AI Level 2 評分標準與應完成項目，再請 AI 根據現有 Level 1 程式碼分析可延伸的方向。接著進一步要求 AI 將 Level 2 的放寬 assumptions 轉換為 notation、限制式與 objective function，並協助實作對應程式。完成初版後，再透過多次 prompt 要求 AI 調整輸出格式，使最終資料夾結構符合作業規定，並整理 demo 時可能被詢問的問題與回答方式。

AI 輔助的主要價值在於加速規格理解、程式架構分析與報告文字整理。由於 Level 2 需要同時處理模型建構、程式實作、JSON 輸出格式與效能分析，AI 能協助將需求拆成較小的步驟，並檢查各部分是否和作業規格一致。不過，AI 產生的內容仍需要人工確認，例如確認是否符合「不可新增決策變數」、輸出 JSON 是否符合指定格式，以及動態排程方法是否能合理對應評分標準。透過人工檢查與多次修正，最後完成可執行且符合 Level 2 要求的進階動態排程程式。

心得：

對於 task_generator:

在這次的即時系統專案中，開發 task_generator 讓我深刻體會到數學約束與演算法設計的博弈。一開始面對任務總數、週期範圍、電力上限、非搶占限制與至少 20% 緊湊任務等諸多硬性指標時，我直覺想用純隨機碰撞再進行大迴圈過濾，卻發現程式因大魔王 Frame Size 公式 ($2f - \gcd(f, \text{period}_j) \leq \text{deadline}_j$) 而陷入無窮迴圈。這是因為當截止時間極短時，隨機產生的週期若無法與 Frame Size 整除，公式在數學上便絕對無法成立。為了解決這個衝突，在與 AI 討論後我改用「由答案逆推」的導向型策略，根據除盡 72 且執行時間小於等於 4 的限制，直接將 Frame Size 鎖死在黃金數字 4，並故意將週期候選池限縮為 4 的倍數，從根本上閹割大魔王公式的殺傷力，讓超緊湊任務能百分之百通過。此外，在處理電力需求 ($6 \leq w_j \leq 18$ 且至少兩個 ≥ 14) 的限制時，我放棄了最初手動硬改特定工作電力值、容易導致統計偏誤的作法，改用「整組重抽」的過濾器機制。得益於逆推優化，生成器能在 0.115 秒內完成運算，這讓我們有資本追求真正的隨機分佈，讓高電力工作隨

機出現，大幅提升了測資對於後續排程與能源優化演算法的測試強健性，也為我接下來實作時間切片與隨機任務插入打下了扎實的信心基礎。

對於 level 1 scheduler:

本次專案中，我主要針對 scheduler.py 與 evaluator.py 進行設計與實作。scheduler.py 是整個系統的核心，因為它不只是單純把 jobs 放進 72 小時的時間軸中，還必須同時考慮 release time、execution time、deadline、preemptive / non-preemptive、frame size、sporadic acceptance test，以及供電設備的容量與電量限制。透過這次實作，我發現即時系統排程並不是只要「有空格就放進去」這麼簡單，而是每一次排入 job 都可能影響後續 jobs 的可行性，尤其 sporadic jobs 到達時，又不能任意移動原本的 periodic schedule，因此 acceptance test 的設計變得非常重要。

在 scheduler 的設計上，我先確保 periodic jobs 能夠形成合法的日前固定排程，再處理 sporadic 與 aperiodic jobs。Sporadic jobs 屬於 hard deadline jobs，因此需要經過 acceptance test，確認在不破壞既有排程的前提下，是否能在 deadline 前完成；aperiodic jobs 則屬於 soft deadline jobs，因此可以等待空檔安排，若逾期則記錄 miss 與 tardiness。這讓我更清楚 hard deadline 和 soft deadline 在實作上的差異：hard deadline 的重點是「不能錯」，soft deadline 的重點是「盡量不要錯，錯了要能量化損失」。

evaluator.py 則讓整個系統的結果變得可以被檢查與說明。剛開始我會比較容易只看 scheduler 執行成功與否，但後來發現「程式有輸出」不代表「排程一定正確」。因此 evaluator 的功能非常重要，它會重新讀取 schedule result，檢查 hard deadline miss rate、soft deadline miss rate、response time、tardiness、jitter、sporadic value rate、傳統機組成本、售電收益與 objective value，也會檢查供需平衡、generator output、renewable output、battery SOC 等限制式。透過 evaluator，我可以比較有信心地確認 scheduler 的輸出不是只有格式正確，而是真的符合限制條件。

在與 AI 協作的過程中，我主要使用 AI 協助釐清作業規格、拆解 scheduler 與 evaluator 的責任、檢查 JSON 格式與路徑問題，以及討論 acceptance test 和三個目標函數的實作方向。我採用的 prompt 策略是逐步詢問，而不是一次要求 AI 完成整份作業。當我遇到問題時，會提供目前的程式檔、輸出結果或錯誤畫面，讓 AI 幫我判斷問題可能出在哪裡，再由我確認是否符合課堂規格。

這次最大的收穫是，我更理解即時系統中的「可行性」和「最佳化」是兩個不同層次。Level 1 的重點是先建立一個穩定、可驗證、符合限制式的排程系統，而不是一開始就追求全域最佳解。

Scheduler 負責產生盡可能好的排程，Evaluator 則負責驗證這個排程是否真的可靠。兩者搭配後，系統才能不只完成排程，也能用數據說明排程品質。

對於 level 2 advanced scheduler:

這次 Level 2 的實作讓我更理解即時系統排程和能源調度結合時的複雜度。Level 1 主要是在已知且固定的條件下完成 periodic jobs、sporadic jobs 和 aperiodic jobs 的排程，但 Level 2 進一步要求考慮實際再生能源出力、即時價格與儲能設備使用成本，讓排程問題不只是「能不能排完」，還要考慮

在動態環境下是否仍然可行，以及不同目標之間的取捨。

在設計 `advanced\_scheduler.py` 時，我們沒有重新寫一套完全不同的排程器，而是選擇在既有 Level 1 scheduler 外面加上一層進階動態排程邏輯。這樣做的好處是可以保留原本已經驗證過的 periodic scheduling、sporadic acceptance test、aperiodic scheduling 和 evaluator 檢查流程，同時把 Level 2 的 actual renewable availability、real-time price、storage aging cost 和 sell shortfall penalty 加入模型中。這也讓我理解到，在系統設計上不一定每次都要重寫全部功能，能夠在穩定的基礎上延伸，有時反而比較可靠。

這次實作中比較困難的部分是理解「動態排程」到底要呈現什麼。原本我以為只要把數值改成另一組輸入即可，但後來在 demo 時經助教提醒，Level 2 更重視的是系統如何面對狀態變化。因此我們採用 event-triggered rolling refresh 的概念，當再生能源實際出力和預測差距過大、即時價格變動過大，或到達固定 checkpoint 時，就更新 renewable availability 與 price score。雖然目前實作是以可重現的模擬資料代表即時資訊，還不是完整連接真實 API 的線上排程系統，但它已經能表現出 Level 2 所要求的動態資訊處理與排程更新概念。

另一個收穫是更清楚了解不同目標之間的 tradeoff。若系統想提高 sporadic jobs 的接納率或維持 hard deadline feasibility，可能需要增加傳統機組發電或使用更多儲能設備，導致 generator cost 和 storage aging cost 上升；若想增加售電收益，又可能減少 reserve，影響突發 jobs 的處理能力。這讓我體會到排程結果不能只看單一指標，而是要同時觀察 deadline miss rate、sporadic value rate、generator cost、market revenue、storage aging cost、sell commitment penalty 和 constraint violations。整體而言，Level 2 的實作讓我從單純的靜態排程，進一步思考如何在不確定性與即時狀態變化下維持系統可行性。透過這次作業，我對 real-time scheduling、energy dispatch、acceptance test、constraint checking，以及成本與收益之間的權衡有了更完整的理解。

六、 針對評分標準 4-1,4-2 說明

4-1 Acceptance Test 方法設計說明：

本程式在 periodic jobs 已完成初始排程後，才對 sporadic jobs 進行 acceptance test。sporadic job 被視為 hard-deadline job，因此只有在不移動既有已接受工作、且能在 release time 到 absolute deadline 之間找到足夠執行時段時，才會被接受。

實作流程如下：

先用 candidate_allocations() 產生 sporadic job 的可行時間組合。若 job 可搶佔，會列出 release 到 deadline 間所有長度為 execution time 的時間組合；若不可搶佔，則只允許連續時段。

用 scheduled_load_by_hour() 計算目前已排入的 periodic jobs 與已接受 sporadic jobs 在每個小時的負載。

在 insert_job_without_moving_existing() 中檢查候選時段是否滿足： $\text{load}[t] + \text{job.energy} \leq \text{load_capacity}[t] - \text{reserve}[t]$ ：優先保留 sporadic reserve；

若無法保留 reserve，才檢查 $\text{load}[t] + \text{job.energy} \leq \text{load_capacity}[t]$ ，代表可使用保留容量但不能超

過總容量。

若找到可行時段，該 sporadic job 被 accept，並把 scheduled_times 寫入 job，加入後續排程狀態。若找不到任何可行時段，該 job 被 reject，理由記錄為「deadline 前沒有不移動既有工作的可行時段」。

最後 build_schedule_result() 重新建立完整能源排程，並由 verify_schedule() / evaluator 檢查 energy balance、renewable cap、generator output/ramp/min-up/min-down、storage charge/discharge、SOC transition 與 SOC bounds。

相關程式位置：

scheduler_opt.py (line 350) 的 insert_job_without_moving_existing() 負責 accept/reject 判斷；

scheduler_opt.py (line 912) 的 run_sporadic_acceptance_test() 負責逐一處理 sporadic jobs；

scheduler_opt.py (line 754) 之後的 verify_schedule() 檢查 deadline 與資源限制。

4-2 Accept / Reject 判斷合理性

本次 acceptance log 中共有 2 個 sporadic jobs，兩個都被接受，且 evaluator 顯示

hard_deadline_miss_rate = 0.0、sporadic_value_rate = 1.0、constraint_violation_count = 0，表示接受後沒有造成 hard deadline miss 或能源限制違反。

Sporadic job 判斷 原因

s1 Accept s1 release time 為 1，absolute deadline 為 10，需要 3 個時段、每時段 energy 15。程式找到 [3, 9, 10] 作為執行時段，全部在 deadline 前，且理由為 accepted: feasible before deadline while preserving reserve，表示接受後仍保留資源餘裕。

s2 Accept s2 release time 為 3，absolute deadline 為 17，需要 4 個時段、每時段 energy 10。程式找到 [11, 12, 13, 14]，完成時間 14 小於 deadline 17，也沒有超過 load capacity 或破壞既有排程，因此接受合理。

若未來有 sporadic job 被拒絕，拒絕條件會是：在 release time 到 deadline 之間找不到足夠空檔，或加入後會使某些小時的負載超過 load_capacity / reserve，進一步可能導致既有 periodic/sporadic hard deadline、energy balance、SOC 或 generator constraints 違反。

七、 評分標準 6(日前保留策略效能分析)說明

6-1 保留策略演算法說明

本系統的日前排程先建立 72 小時的 periodic jobs 固定排程，並在不破壞 periodic jobs deadline 與能源限制式的前提下，預留可插入 sporadic jobs 的資源餘裕。保留策略的核心是在每個時間點計算 load_capacity[t]，並保留一部分 capacity 作為 sporadic reserve。程式中設定：

reserve[t] = min(12.0, load_capacity[t] * 0.08)

也就是每小時最多保留 12 MWh，或保留該小時可用容量的 8%。這個 reserve 不會直接寫成新的輸

出欄位，而是在 acceptance test 時作為判斷門檻使用。

當 sporadic job 到達時，系統先產生該 job 在 release time 到 absolute deadline 之間的所有候選執行時段。若 job 是 preemptive，允許分散安排；若是 non-preemptive，則只允許連續時段。接著系統依照目前已排程 jobs 的每小時負載，檢查：

$$\text{load}[t] + \text{job.energy} \leq \text{load_capacity}[t] - \text{reserve}[t]$$

若成立，代表該 sporadic job 可在 deadline 前完成，而且接受後仍保留 reserve，因此直接接受。若無法保留 reserve，系統才進一步檢查：

$$\text{load}[t] + \text{job.energy} \leq \text{load_capacity}[t]$$

這代表可使用保留容量，但仍不能超過總資源上限。若兩種檢查都失敗，則拒絕該 sporadic job，避免破壞既有 periodic jobs、已接受 hard-deadline jobs 或能源限制。

本次 Level 1 實際結果顯示，保留策略成功讓兩個 sporadic jobs 都被接受，且都在 hard deadline 前完成。s1 被安排在 [3, 9, 10]，deadline 為 10；s2 被安排在 [11, 12, 13, 14]，deadline 為 17。兩者 acceptance reason 都是 accepted: feasible before deadline while preserving reserve，表示它們不只可行，還沒有用盡保留餘裕。

同時，系統最後仍維持可行：hard_deadline_miss_rate = 0.0、constraint_violation_count = 0、verification_passed = true。這表示 reserve 策略沒有造成既有 hard deadline jobs 延遲，也沒有破壞能源平衡、SOC、generator constraints 等系統限制。

6-2 目標函數權衡分析

本系統的三個主要目標為：最小化 aperiodic miss、最小化傳統機組成本、最大化售電收益。Level 1 的實際結果如下：

$$\text{soft_deadline_miss_rate} = 0.0$$

$$\text{soft_deadline_miss_count} = 0$$

$$\text{sporadic_value_rate} = 1.0$$

$$\text{generator_cost} = 260442.0$$

$$\text{market_revenue} = 331650.2$$

$$\text{objective_value} = -71208.2$$

$$\text{constraint_violation_count} = 0$$

從結果可看出，排程優先確保 hard deadline jobs 與 sporadic jobs 的可行性，再利用剩餘容量安排 aperiodic jobs 與售電。三個 aperiodic jobs 全部在 soft deadline 前完成：a1 完成於 t=3，deadline=15；a2 完成於 t=6，deadline=12；a3 完成於 t=8，deadline=16，因此 soft miss 為 0，沒有產生 aperiodic miss penalty。

不過，保留 reserve 會帶來目標函數上的取捨。為了讓 sporadic jobs 有機會在到達後插入，日前排程不能把所有低成本時段或高收益售電時段完全用滿。這可能降低部分售電彈性，或使某些工作安排到傳統機組成本較高的時段。但本次結果中，sporadic value rate 達到 1.0，表示所有 sporadic execution time 都在 hard deadline 前完成；同時 market revenue 為 331650.2，高於 generator cost

260442.0，使 objective value 為 -71208.2，代表整體仍有良好的經濟效益。

因此，本次 Level 1 排程在三個目標之間取得平衡：保留策略提高 sporadic 接納能力，aperiodic miss 維持 0，且售電收益仍能抵銷傳統機組成本。這也顯示 reserve 並非越多越好；若 reserve 過大，可能犧牲售電收益或增加傳統機組成本；若 reserve 過小，sporadic jobs 可能被拒絕，導致 sporadic value rate 下降。本次使用 8% 且上限 12 MWh 的 reserve 設定，在此測資下達到 sporadic_value_rate = 1.0 與 constraint_violation_count = 0。

八、 評分標準第 8 項之說明

8-1 進階排程方法設計

Level 2 採用 event-triggered rolling refresh 動態排程方法。此方法以 Level 1 靜態排程為基礎，但在排程過程中加入實際再生能源出力、即時市場價格、儲能老化成本與 day-ahead sell commitment penalty。當系統偵測到狀態變化時，會重新更新可用能源、價格評分與排程決策，再重新檢查 hard deadline jobs 與 sporadic acceptance test。

本方法的更新時機包含三種：

1. rolling checkpoint : $t = 1, 24, 48$
2. renewable shortfall > 5 MWh
3. real-time price change > 10 \$/MWh

本次 Level 2 結果共有 dynamic_update_count = 25 次更新。例如 $t=8$ 到 $t=14$ 發生 renewable shortfall， $t=21$ 到 $t=23$ 發生 real-time price update， $t=1$ 、24、48 則是固定 rolling checkpoint。

選擇此方法的理由是：不必每一小時都完整重排，而是在再生能源或價格明顯改變時才更新，因此可降低重排頻率與計算成本。同時，它能在 renewable shortfall 發生時補足供電，在即時價格變化時重新調整售電策略。缺點是不同目標會互相衝突，例如提高 sporadic 接納率與維持 hard deadline 可行性，可能需要增加傳統機組出力，導致 generator cost 上升；追求即時售電收益，也可能增加 sell commitment shortfall penalty 或儲能 throughput。

```
"dynamic_model": {  
  "method": "event-triggered rolling refresh with actual renewable output and real-time prices",  
  "storage_aging_cost_per_mwh": 6.0,  
  "sell_shortfall_penalty_per_mwh": 35.0  
},
```

```

"dynamic_update_log": [
  {
    "hour": 1,
    "forecast_renewable_mwh": 0.0,
    "actual_renewable_mwh": 0.0,
    "renewable_gap_mwh": 0.0,
    "day_ahead_price": 84.0,
    "realtime_price": 84.0,
    "trigger_reasons": [
      "rolling horizon checkpoint"
    ],
    "action": "refresh scores, recompute feasible dispatch, then re-run acceptance decisions for unreleased soft/dynamic jobs"
  },
  {
    "hour": 8,
    "forecast_renewable_mwh": 22.4,
    "actual_renewable_mwh": 16.128,
    "renewable_gap_mwh": 6.272,
    "day_ahead_price": 93.0,
    "realtime_price": 93.0,
    "trigger_reasons": [
      "renewable shortfall"
    ],
    "action": "refresh scores, recompute feasible dispatch, then re-run acceptance decisions for unreleased soft/dynamic jobs"
  },
],

```

```

{
  "hour": 21,
  "forecast_renewable_mwh": 0.0,
  "actual_renewable_mwh": 0.0,
  "renewable_gap_mwh": 0.0,
  "day_ahead_price": 86.0,
  "realtime_price": 101.48,
  "trigger_reasons": [
    "real-time price update"
  ],
  "action": "refresh scores, recompute feasible dispatch, then re-run acceptance decisions for unreleased soft/dynamic jobs"
},

```

8-2 排程結果正確性說明

Level 2 最終輸出顯示排程結果維持可行：

verification_passed = true

constraint_violation_count = 0

hard_deadline_miss_rate = 0.0

soft_deadline_miss_rate = 0.0

sporadic_value_rate = 1.0

```

C: > C program > Submission_5 > output > {} evaluation_results.json > .
1  {
2    "hard_deadline_miss_rate": 0.0,
3    "hard_deadline_miss_count": 0,
4    "hard_deadline_job_count": 33,
5    "soft_deadline_miss_rate": 0.0,
6    "soft_deadline_miss_count": 0,
7    "soft_deadline_job_count": 3,
8    "average_tardiness": 0,

```

所有 periodic 與 accepted sporadic hard deadline jobs 都在期限前完成。

sporadic jobs 共 2 個，全部接受：

s1: release=1, deadline=10, scheduled_times=[3, 9, 10]

s2: release=3, deadline=17, scheduled_times=[11, 12, 13, 14]

兩者 reason 都是 accepted: feasible before deadline while preserving reserve，表示更新後仍能保留資源餘裕。

```
{
  "level": 2,
  "sporadic": [
    {
      "job_id": "s1",
      "type": "sporadic",
      "release_time": 1,
      "absolute_deadline": 10,
      "execution_time": 3,
      "energy": 15.0,
      "accepted": true,
      "scheduled_times": [
        3,
        9,
        10
      ],
      "reason": "accepted: feasible before deadline while preserving reserve",
      "level2_policy": "hard-deadline acceptance test after refreshing renewable and price state"
    },
    {
      "job_id": "s2",
      "type": "sporadic",
      "release_time": 3,
      "absolute_deadline": 17,
      "execution_time": 4,
      "energy": 10.0,
      "accepted": true,
      "scheduled_times": [
        11,
        12,
        13,
        14
      ],
      "reason": "accepted: feasible before deadline while preserving reserve",
      "level2_policy": "hard-deadline acceptance test after refreshing renewable and price state"
    }
  ],
}
```


Aperiodic jobs 共 3 個，也全部在 soft deadline 前完成：

a1: completion=3, deadline=15, miss=false

a2: completion=6, deadline=12, miss=false

a3: completion=8, deadline=16, miss=false

```
38     "aperiodic": [  
39         {  
40             "job_id": "a1",  
41             "type": "aperiodic",  
42             "release_time": 1,  
43             "absolute_deadline": 15,  
44             "execution_time": 3,  
45             "energy": 15.0,  
46             "scheduled_times": [  
47                 1,  
48                 2,  
49                 3  
50             ],  
51             "completion_time": 3,  
52             "miss": false,  
53             "tardiness": 0,  
54             "reason": "scheduled before soft deadline",  
55             "level2_policy": "soft-deadline queue scheduled with refreshed state and price-aware scoring"  
56         },  
57         {  
58             "job_id": "a2",  
59             "type": "aperiodic",  
60             "release_time": 3,  
61             "absolute_deadline": 12,  
62             "execution_time": 4,  
63             "energy": 10.0,  
64             "scheduled_times": [  
65                 3,  
66                 4,  
67                 5,  
68                 6  
69             ],  
70             "completion_time": 6,  
71             "miss": false,  
72             "tardiness": 0,  
73             "reason": "scheduled before soft deadline",  
74             "level2_policy": "soft-deadline queue scheduled with refreshed state and price-aware scoring"  
75         }  
76     ]  
77 }
```

```
    "job_id": "a3",  
    "type": "aperiodic",  
    "release_time": 5,  
    "absolute_deadline": 16,  
    "execution_time": 4,  
    "energy": 12.0,  
    "scheduled_times": [  
        5,  
        6,  
        7,  
        8  
    ],  
    "completion_time": 8,  
    "miss": false,  
    "tardiness": 0,  
    "reason": "scheduled before soft deadline",  
    "level2_policy": "soft-deadline queue scheduled with refreshed state and price-aware scoring"  
}
```

沒有 job rejection、沒有 miss deadline，也沒有 constraint violation。

系統限制方面，schedule_result.json 的 verification 為 passed: true 且 errors: []，evaluation 中 constraint_violations 也是空陣列，表示能源平衡、generator constraints、renewable output limit、SOC transition、SOC bounds 與充放電限制皆維持可行。

Level 2 雖然沒有造成 job rejection 或 miss deadline，但因為實際 renewable output 低於 forecast，並加入 storage aging cost 與 sell shortfall penalty，因此成本上升。

8-3 排程結果比較

Level 1 與 Level 2 的主要結果如下：

Level 1:

```
generator_cost = 260442.0
market_revenue = 331650.2
objective_value = -71208.2
soft_deadline_miss_rate = 0.0
sporadic_value_rate = 1.0
```

Level 2:

```
generator_cost = 315272.0
market_revenue = 376847.64224
storage_aging_cost = 3128.988
sell_commitment_penalty = 4391.1
objective_value = -54055.55424
soft_deadline_miss_rate = 0.0
sporadic_value_rate = 1.0
```

```
=== Level 2 Advanced Scheduler ===
hard deadline miss rate: 0.0
soft deadline miss rate: 0.0
sporadic value rate: 1.0
generator cost: 315272.0
market revenue: 376847.64224
storage aging cost: 3128.988
sell commitment penalty: 4391.1
objective value: -54055.55424
constraint violations: 0
```

```
hard deadline miss rate: 0.0
soft deadline miss rate: 0.0
average response time: 4.166667
average tardiness: 0
sporadic value rate: 1.0
generator cost: 260442.0
market revenue: 331650.2
objective value: -71208.2
constraint violations: 0
```

```
"comparison_to_level1": {
  "generator_cost_delta": 54830.0,
  "market_revenue_delta": 45197.44224,
  "objective_value_delta": 17152.64576,
  "sporadic_value_rate_delta": 0.0,
  "soft_deadline_miss_rate_delta": 0.0
},
```

與 Level 1 相比，Level 2 的 generator_cost 增加 54830.0。主要原因是實際再生能源低於日前預測時，系統必須用更多傳統機組補足供電，因此發電成本上升。

Level 2 的 market_revenue 增加 45197.44224，原因是即時市場價格在部分時段高於日前價格，動態排程能利用 real-time price 重新計算售電收益。不過 Level 2 額外加入 storage_aging_cost = 3128.988 與 sell_commitment_penalty = 4391.1，因此雖然售電收益提高，最終 objective value 仍比 Level 1 高 17152.64576，代表整體成本效益變差。

在服務品質方面，兩者都維持：

soft_deadline_miss_rate = 0.0

sporadic_value_rate = 1.0

constraint_violation_count = 0

因此 Level 2 的動態方法成功維持 job deadline 與系統可行性，但代價是傳統機組成本、儲能老化成本與售電承諾違約成本增加。這說明動態排程能提升面對不確定性的穩定性，但不保證三個目標函數同時變好。

九、 排程結果格式說明 和 評估結果格式說明：

G. 排程結果格式說明

本作業 Level 2 仍使用與 Level 1 相同的 schedule_result.json 輸出格式，保留每個時間點的必要欄位：

t

P

k

sell

soc

missed_aperiodic

rejected_sporadic

其中 P 表示每台發電設備在該時間點的供電量，k 表示各 job 從各設備取得的電能分配，sell 表示售電量，soc 表示儲能設備在該時段結束後的電量狀態。missed_aperiodic 用來記錄 soft deadline miss 的 aperiodic jobs，rejected_sporadic 用來記錄 acceptance test 未通過的 sporadic jobs。

Level 2 在不刪除必要欄位的前提下，額外加入：

level

dynamic_model

dynamic_update_log

verification

level = 2 表示此排程結果由 Level 2 進階動態排程方法產生。dynamic_model 說明使用的動態排程方法、儲能老化成本參數與售電 shortfall penalty。dynamic_update_log 記錄每次動態更新的時間點、觸發原因，例如 renewable shortfall、real-time price update 或 rolling checkpoint。verification 則

記錄最終排程是否通過限制式檢查。

本次 Level 2 最終結果中，`verification.passed = true` 且 `errors = []`，表示排程結果滿足主要限制式，包括 `hard deadline`、能源平衡、`generator constraints`、`renewable output limit`、`SOC transition` 與儲能上下限。

H. 評估結果格式說明

Level 2 的 `evaluation_results.json` 也沿用 Level 1 的基本評估欄位，例如：

`hard_deadline_miss_rate`

`soft_deadline_miss_rate`

`average_tardiness`

`max_tardiness`

`average_response_time`

`max_response_time`

`completion_time_jitter`

`acceptance_test`

`sporadic_value_rate`

`generator_cost`

`market_revenue`

`objective_value`

`constraint_violation_count`

這些欄位用來評估 `deadline` 表現、`response time`、`tardiness`、`sporadic acceptance` 效能、傳統機組成本、售電收益與限制式違反情形。

因為本作業有實作 Level 2，因此額外加入以下欄位：

`storage_throughput_mwh`

`storage_aging_cost`

`sell_commitment_shortfall_mwh`

`sell_commitment_penalty`

`level2_relaxed_assumptions`

`comparison_to_level1`

`verification_passed`

其中 `storage_throughput_mwh` 與 `storage_aging_cost` 用來評估儲能設備因充放電造成的老化成本；

`sell_commitment_shortfall_mwh` 與 `sell_commitment_penalty` 用來計算 Level 2 中 `day-ahead sell`

`commitment` 未達成時的懲罰成本。`level2_relaxed_assumptions` 說明本次放寬的假設，包括實際再生能源出力不同於預測值、儲能老化成本、即時市場價格與售電承諾懲罰。`comparison_to_level1` 則提供 Level 2 與 Level 1 在 `generator cost`、`market revenue`、`objective value`、`sporadic value rate` 與 `soft deadline miss rate` 上的差異。

本次 Level 2 評估結果顯示：

hard_deadline_miss_rate = 0.0

soft_deadline_miss_rate = 0.0

sporadic_value_rate = 1.0

constraint_violation_count = 0

verification_passed = true

代表 Level 2 動態排程雖然考慮了更真實的不確定性與成本，但仍維持所有 hard deadline jobs 準時完成、aperiodic jobs 無 miss、sporadic jobs 全部接受，且沒有系統限制式違反。